Sentiment Analysis on Amazon Food Reviews using Natural Language Processing

GitLab - Code

1. Contribution & Roles Table

Team Member	Project Role	Core Responsibilities & Deliverables
Vaibhav Murlidhar Thalal	Data Preprocessing EDA Report	Cleaned the raw dataset by removing duplicates, fixing missing values, and sorting the data by timestamp. Also explored the data to check for class imbalances and investigate the "Helpfulness" metric.
Henry Unachukwu	Basic Visualization Evaluation Transformers Report	Created bar charts and histograms to show how the target variables are distributed. Implemented the pre-trained DistilBERT pipeline and compared model performance using Accuracy, F1-Score and AUC.
Avinash Vadlamudi	Text preprocessing MNB Model Report	Normalized, tokenized and removed stopwords from the raw text data. Then, trained and tuned a Multinomial Naive Bayes classifier to use as a baseline.
Jaswanth Vajrapu	Feature Extraction Log model Report	Converted the cleaned text into numerical vectors using the TF-IDF method. After that, trained and tuned a Logistic Regression model.

Table 1: Project Roles and Responsibilities

2. Abstract

This project looks at sentiment classifying Amazon Fine Food Reviews as positive or negative using Logistic Regression, Multinomial Naive Bayes and DistilBERT-based transformer model. After converting scores into positive(1) or negative(0) labels, we evaluated model accuracy. Logistic Regression performed best with an accuracy of 0.89, followed by Naive Bayes at 0.82 and the transformer at 0.83. These results highlight the advantages and limitations of traditional machine learning approaches compared to transformer-based models.

3. Problem & Data

Problem Definition

The goal of this project is to create a binary sentiment classifier that labels Amazon reviews as positive if the rating is above three and negative if it is three or below. We mainly use accuracy to measure model performance, with AUC for class separation and F1-score to address class imbalance.

Data Description

The dataset has 568k+ Amazon food reviews sourced from Kaggle. We cleaned it by removing duplicates, resolving duplicates using timestamps, imputing missing helpfulness ratios and applying NLP preprocessing steps such as tokenization and stop-word removal. The data includes both structured and unstructured data, with text(review) as the main modelling feature and the score column used to create sentiment labels.

4. Methodologies

EDA and Preprocessing

We did a brief EDA to check data quality, distribution and key patterns in the reviews. The dataset showed a strong bias toward positive ratings and needed cleaning to remove duplicates and missing values for reliable modelling.

Key observations:

- Most reviews were rated 4 or 5, indicating highly skewed positive feedback
- Exact and subset duplicates were removed.
- Visualizations such as score distributions and word clouds highlighted common themes in positive and negative reviews

Figure 1: Basic Visualizations

Text Preprocessing

We used NLP techniques to structure the raw text into useful features:

1. **Normalization:** Text was converted to lowercase to ensure consistency.
    a. **What it does:** Converts "Delicious", "DELICIOUS" and "delicious" to just "delicious".
    b. **Why it matters:** This step reduces the **dimensionality** of data. We want the model to treat the same word, no matter how it is capitalized.

```
review = "I honestly LOVED the chips, but the packaging was terrible!"
normalized_review = review.lower()
normalized_review

'i honestly loved the chips, but the packaging was terrible!'
```

Figure 2: Normalization

2. **Tokenization**: Splits a sentence into individual units called tokens (words).
    a. **What it does:** "Not good" becomes ["Not", "good"].
    b. **Why it matters:** Tokenization changes unstructured text into structured data by splitting it into lists of words.

```
words = nltk.word_tokenize(normalized_review)
words

['i',
 'honestly',
 'loved',
 'the',
 'chips',
 ',',
 'but',
 'the',
 'packaging',
 'was',
 'terrible',
 '!']
```

Figure 3: Tokenization

3. **Stopword Removal:** NLTK's English stopword list was applied to remove non-informative tokens.
    a. **What it does:** Removes extremely common words like "the", "is", "at", "which".
    b. **Why it matters:** These words show up in every review whether positive or negative. They do not carry sentiment and use up memory. Removing them helps the model focus on important words like "tasty" or "rotten."

```
filtered_words = [word for word in words if word not in stop_words]
filtered_words

['honestly', 'loved', 'chips', ',', 'packaging', 'terrible', '!']
```

Figure 4: Stopword Removal

4. **Punctuation Removal**: This step removes tokens that do not help with prediction.
    a. **What it does:** Removes punctuations like **!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~**
    b. **Why it matters:** Removing punctuation is important for cleaning text data and helps ML models find patterns more easily.

```
clean_tokens = [word for word in filtered_words if word not in string.punctuation]
clean_tokens

['honestly', 'loved', 'chips', 'packaging', 'terrible']
```

Figure 5: Punctuation Removal

Final Outcome: We began with a 10-word sentence full of noise and finished with 5 keywords that clearly capture the sentiment (Love or Terrible) and the subject (Chips or Packaging).

Feature Extraction

Vectorization: We used **TF-IDF** to transform text into numerical vectors, giving more weight to unique words than common ones.

It calculates a score for every word in every review based on two factors:

1. **TF (Term Frequency):** How often does the word appear in this specific review?
2. **IDF (Inverse Document Frequency):** How rare is this word across all 500,000 reviews?

TF - IDF score = TF * IDF

```
cleaned_reviews = ["honestly chips packaging terrible",
                   "tea fantastic terrible"]

tfidf_matrix = vectorizer.fit_transform(cleaned_reviews)
print(vectorizer.get_feature_names_out())
print(tfidf_matrix.toarray())
```

```
['chips' 'fantastic' 'honestly' 'packaging' 'tea' 'terrible']
[[0.53404633 0.         0.53404633 0.53404633 0.         0.37997836]
 [0.         0.6316672  0.         0.         0.6316672  0.44943642]]
```

Figure 6: TF-IDF Matrix Score

5. Models & Validation

We compared three model types:

1. **Logistic Regression** :
   Selected as the baseline due to
   a. Strong performance on sparse, high-dimensional data
   b. Interpretability via model coefficients
2. **Multinomial Naive Bayes :**
   A probabilistic model is chosen for
   a. It's speed
   b. Effectiveness with high-dimensional text data.
3. **DistilBERT :**
   A modern contextual language model, implemented via Hugging Face's pipeline
   a. Model: distilbert-base-uncased-finetuned-sst-2-english
   b. Provides contextual embeddings
   c. Captures semantics (e.g., "not great" vs "great")

Validation Protocol

1. **80/20 train/test split**
2. The **TF-IDF vocabulary was fitted before splitting**
   a. ensures identical feature dimensions in train & test
   b. reduces leakage risk while improving stability of sparse features
3. Evaluation metrics included:
   a. **Accuracy**
   b. **Precision, Recall, F1-score**
   c. **Confusion Matrix**
   d. **AUC Score**

6. Results

Model	Overall Accuracy	Neg-(0) F1-Score	Pos+(1) F1-Score	AUC Score	Training Time(s)
Logistic Regression	0.8900	0.75	0.94	0.9416	16.91s
Multinomial Naive Bayes	0.8219	0.34	0.90	0.9101	0.34s
Transformer	0.8344	0.69	0.89	0.8311	3678.49s

Table 2: Summary of Results

Best Performer: The Logistic Regression model had the highest accuracy (89%) and the best balance between precision and recall (Negative F1: 0.75). It separated the classes well, with a high AUC of 0.94.

The Multinomial Naive Bayes model was the fastest model (0.34 sec), making it ideal for low-latency uses. However, it struggled with negative reviews (Negative F1: 0.34), often classifying valid complaints as positive.

The Transformer model had a good accuracy (83%) but took much longer to train (about 1 hour compared to 17 seconds for Logistic Regression). In this case, the deep learning model did not justify the extra resource consumption compared to the well-tuned linear baseline.

7. Analysis & Discussion

Interpretation*:* Logistic Regression had the highest accuracy (89%) and best AUC (0.94), showing it worked well to the sentiment division. Naive Bayes performed worse on negative reviews (F1 = 0.34), which shows imbalance-sensitivity. The transformer model showed positive reviews but needed much more computing power. Overall, the simpler linear models outperformed the complex model because of the dataset and the way we mapped the scores.

Error Analysis: The transformer model made mistakes mostly on short reviews that lacked context. Some negative reviews with polite wording, such as "not great but okay/Just what I needed, another problem", confused all three models.

Limitations*:* The dataset is heavily skewed toward positive scores (Score=5). Even though we checked performance for each score group, the model still tends to predict positive results. Also, our preprocessing did not handle sarcasm, negation, or aspect-based sentiment.

8. Ethical & Legal Considerations

The dataset was sourced from Kaggle, which republishes Amazon's publicly available review data. Because it only has text reviews and star ratings, with no personal information, the privacy risk is very low.

9. Conclusion & Future Work

Sentiment analysis means identifying the feeling or emotion in a text or document. There are many ways to do this. In this project, we used several machine learning algorithms and different feature extraction methods. Logistic Regression gave the best results. Since we only compared three models, future work could include trying other techniques like CNNs, SMOTE or new visualization methods. We could also build a model that does sentiment analysis based on specific features in a product.

10. Bibliography

1. McAuley, J., & Leskovec, J. (2013). *Amazon Fine Food Reviews.* Stanford Network Analysis Project. [Available at Kaggle]
2. 'HuggingFace's Transformers: State-of-the-art Natural Language Processing.'[Link]
3. S. Choudhary and C. Chhabra (2021), 'Sentiment Analysis of Amazon Food Review Data,' Fourth International Conference on Computational Intelligence and Communication Technologies[Link]
4. GeeksforGeeks. (2025). *Multinomial Naive Bayes.* [Online]
5. DataCamp (2025), *Natural Language Processing (NLP) in Python.* Interactive Course, Updated Jul. 2025. [Online].